

Documentos Anidados y Colecciones Relacionadas en MongoDB Compass

Documentos anidados

Los documentos anidados en MongoDB son documentos que se encuentran dentro de otros documentos. Permiten estructurar datos de manera jerárquica, lo que puede facilitar la representación de relaciones complejas entre diferentes unidades de datos.

```
{
  "_id": "cliente1",
  "nombre": "Juan Pérez",
  "email": "juan.perez@example.com ",
  "direcciones": [
    {
      "tipo": "Casa",
      "direccion": "Calle Falsa 123",
      "ciudad": "Madrid"
    },
    {
      "tipo": "Trabajo",
      "direccion": "Avenida Siempre Viva 742",
      "ciudad": "Madrid"
    }
  ]
}
```

En este ejemplo, el documento del cliente contiene un array direcciones, donde cada objeto dentro del array representa una dirección relacionada con el cliente. Este enfoque centraliza la información del cliente, pero se debe tener cuidado de no anidar excesivamente si el número de direcciones puede ser muy grande.

Colecciones Relacionadas

En situaciones donde los documentos son independientes o se utilizan en diferentes contextos, es preferible mantener colecciones separadas. Esto se conoce como una relación entre colecciones.

Ejemplo, pedidos y productos:

Colección de productos

```
{
  "_id": "producto1",
  "nombre": "Camiseta",
  "precio": 19.99
}
```

Colección de pedidos

```
{
  "_id": "pedido1",
  "clienteId": "cliente1",
  "productos": [
    {
      "productoId": "producto1",
      "cantidad": 2
    },
    {
      "productoId": "producto2",
      "cantidad": 1
    }
  ],
  "fecha": "2023-10-01"
}
```

En este caso, la colección de pedidos contiene una referencia al clienteId y una lista de productos, que vincula cada producto a su identificación. Este enfoque permite manejar actualizaciones y consultas de manera más eficiente al evitar duplicación de datos.

Análisis y Validación de Datos

Es crucial definir el esquema de datos y validar la información para asegurar que los documentos se ajusten a las reglas y estructuras esperadas. Establecer campos obligatorios ayuda a garantizar que la información crítica esté siempre presente, evitando inconsistencias.

Schema Validation: Utiliza la validación de esquemas para definir restricciones sobre los documentos, como tipos de datos, requerimientos de campo, y patrones específicos.

Ejemplo de Validación de Esquema:

Supongamos que queremos crear un esquema para garantizar que los documentos en la colección productos mantengan ciertos requisitos. El esquema podría definirse de la siguiente manera:

```
{
  $jsonSchema: {
    bsonType: "object",
    required: ["nombre", "precio"],
    properties: {
      nombre: {
        bsonType: "string",
        description: "Debería ser una cadena y es obligatorio."
      },
      precio: {
        bsonType: "decimal",
        description: "Debería ser un número decimal y es obligatorio."
      },
      stock: {
        bsonType: "int",
        description: "Debería ser un número entero pero no es obligatorio."
      }
    }
  }
}
```

Este esquema asegura que cada documento en la colección productos tenga un nombre y un precio, mientras que stock es opcional. La validación de esquemas es una buena práctica para mantener la integridad de los datos.